



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



INSTITUTO TECNOLÓGICO DE ZACATECAS

MAESTRÍA EN SISTEMAS COMPUTACIONALES



INGENIERÍA EN SOFTWARE

PROYECTO FINAL - REPORTE DE ASEGURAMIENTO DE CALIDAD

NOMBRE DEL DOCENTE:

JOSÉ FRANCISCO DE LA ROSA IBARRA

EQUIPO: C&O

INTEGRANTES:

GRETN ODETH CUELLAR ACOSTA

LAURA CORINA ZAPATA LONGORIA

ZACATECAS, ZAC; A MAYO 2026

Contenido

REPORTE DE ASEGURAMIENTO DE CALIDAD	3
1. Resumen.....	3
2. Alcance de las pruebas	3
2.1 Componentes validados con Jest	3
2.2 Componentes validados con k6	4
3. Herramientas y ambiente de ejecución	4
4. Resultados de pruebas unitarias con Jest	5
4.1 Interpretación de suites y casos de prueba	5
4.2 Resultados de cobertura de código	5
5. Resultados de pruebas de rendimiento con k6	6
5.1 Prueba Smoke	6
5.2 Prueba Load.....	7
5.3 Prueba Stress	7
6. Análisis técnico de resultados.....	8
7. Evidencia de comandos ejecutados	8
7.1 Comandos Jest	8
7.2 Comandos k6	9
8. Conclusión y dictamen de calidad	9

REPORTE DE ASEGURAMIENTO DE CALIDAD

1. Resumen

El presente documento concentra los resultados del proceso de aseguramiento de calidad aplicado al backend del sistema de Joyería Maison C.O. Élégance, el objetivo principal fue verificar, mediante pruebas automatizadas, que la lógica principal del sistema funcionara de manera correcta y que la API respondiera adecuadamente bajo distintos niveles de carga.

El proceso de validación se dividió en dos grupos principales: pruebas unitarias con Jest y pruebas de rendimiento con k6. Las pruebas unitarias comprobaron el comportamiento interno de los servicios, middlewares y utilidades del sistema; mientras que las pruebas k6 permitieron evaluar el tiempo de respuesta, estabilidad, porcentaje de fallos y capacidad de atención de peticiones HTTP bajo escenarios smoke, load y stress.

Indicador	Resultado
Estado general del backend	APROBADO
Pruebas unitarias Jest	47 exitosas de 47 ejecutadas
Suites de prueba Jest	6 exitosas de 6 ejecutadas
Cobertura global de statements	99.52%
Cobertura global de branches	92.83%
Cobertura global de functions	100%
Cobertura global de lines	99.50%
Pruebas k6 ejecutadas	Smoke, Load y Stress
Fallos HTTP k6	0.00% en todos los escenarios
Resultado final	CUMPLIDO

2. Alcance de las pruebas

El alcance de las pruebas se centró en validar los componentes más relevantes del backend, principalmente aquellos relacionados con reglas de negocio, control de errores, seguridad básica, operaciones sobre productos, categorías y carrito de compras. Las pruebas fueron diseñadas para comprobar escenarios exitosos y escenarios de error controlado.

2.1 Componentes validados con Jest

Suite de prueba	Propósito de validación	Estado
-----------------	-------------------------	--------

auth.service.test.ts	Validar registro, inicio de sesión, credenciales incorrectas, usuario inexistente y comportamiento de autenticación.	EXITOSO
product.service.test.ts	Validar creación, consulta, actualización, eliminación y manejo de productos inexistentes.	EXITOSO
category.service.test.ts	Validar operaciones relacionadas con categorías y manejo de errores.	EXITOSO
cart.service.test.ts	Validar administración del carrito, cantidades y productos asociados.	EXITOSO
middlewares.test.ts	Validar manejo de errores, respuestas controladas y comportamiento de middleware.	EXITOSO
utils.test.ts	Validar funciones auxiliares reutilizables del backend.	EXITOSO

2.2 Componentes validados con k6

Escenario	Objetivo	Endpoints verificados	Estado
Smoke	Comprobar que la API responde correctamente con carga mínima.	API base, catálogo y categorías.	EXITOSO
Load	Evaluar comportamiento con carga gradual hasta 10 usuarios virtuales.	GET /products, GET /products/:id, GET /categories.	EXITOSO
Stress	Evaluar estabilidad de la API bajo carga alta hasta 50 usuarios virtuales.	Catálogo, detalle de producto y categorías.	EXITOSO

3. Herramientas y ambiente de ejecución

Herramienta	Uso dentro del proceso de QA
Jest	Ejecución de pruebas unitarias automatizadas del backend.
ts-jest	Soporte para ejecutar pruebas escritas en TypeScript.
k6	Ejecución de pruebas de rendimiento, carga y estrés sobre endpoints HTTP.
Node.js	Entorno de ejecución del backend.
Express	Framework utilizado para exponer la API.
Prisma ORM	Capa de acceso a datos del sistema.
PostgreSQL	Base de datos utilizada por el proyecto.

4. Resultados de pruebas unitarias con Jest

Las pruebas unitarias ejecutadas con Jest tuvieron como finalidad validar unidades específicas del sistema de forma aislada. Este enfoque permite verificar que los servicios y componentes internos produzcan resultados correctos sin depender de la ejecución manual del servidor ni de la interacción directa con usuarios finales.

Métrica	Valor obtenido	Interpretación
Test Suites	6 passed, 6 total	Todas las suites o archivos de prueba fueron ejecutados correctamente.
Tests	47 passed, 47 total	Los 47 casos de prueba definidos fueron exitosos.
Snapshots	0 total	No se utilizaron pruebas basadas en snapshots.
Tiempo de ejecución	3.518 s	La ejecución fue rápida y adecuada para integración continua.
Resultado	APROBADO	No se detectaron pruebas fallidas.

4.1 Interpretación de suites y casos de prueba

Una suite de prueba representa un archivo o conjunto de pruebas agrupadas. En este proyecto se ejecutaron 6 suites, correspondientes a los módulos principales del backend. Cada caso de prueba representa una validación específica escrita mediante `test()` o `it()`. Por lo tanto, el total de casos de prueba ejecutados fue de 47.

4.2 Resultados de cobertura de código

En este proyecto, la cobertura fue generada mediante Jest/Istanbul y se obtuvo un resultado global superior al 90% en todas las métricas relevantes.

Métrica	Resultado obtenido	Criterio mínimo	Estado
Statements	99.52%	90%	CUMPLIDO
Branches	92.83%	90%	CUMPLIDO
Functions	100%	90%	CUMPLIDO
Lines	99.50%	90%	CUMPLIDO

- **Statements:** Indica el porcentaje de sentencias de código ejecutadas por las pruebas.
- **Branches:** Mide la cobertura de caminos condicionales, por ejemplo `if`, `else`, `switch` o validaciones alternativas.
- **Functions:** Representa el porcentaje de funciones ejecutadas durante las pruebas.
- **Lines:** Indica el porcentaje de líneas de código cubiertas por los casos de prueba.

El resultado de branches es especialmente importante porque demuestra que no solo se probaron escenarios correctos, sino también caminos alternativos y posibles errores. Al superar el 90% en esta métrica, se considera que el sistema cuenta con una protección sólida frente a cambios futuros.

5. Resultados de pruebas de rendimiento con k6

Además de las pruebas unitarias, se ejecutaron pruebas de rendimiento con k6 para evaluar la respuesta de la API bajo diferentes niveles de carga. Las pruebas k6 se organizaron en tres escenarios: smoke, load y stress.

5.1 Prueba Smoke

La prueba smoke se utiliza como una validación inicial para comprobar que la API se encuentra disponible y que los endpoints principales responden correctamente con una carga mínima. En este caso se ejecutó con 1 usuario virtual durante 20 segundos.

Métrica	Resultado
Comando	npm run k6:smoke
Script	k6/smoke.js
Usuarios virtuales máximos	1 VU
Duración	20 s
Checks totales	80
Checks exitosos	80 de 80 (100.00%)
Checks fallidos	0 de 80 (0.00%)
Peticiones HTTP	60
http_req_failed	0.00%
Duración promedio	4.02 ms
p(95)	5.4 ms
Umbral p(95)	< 1200 ms
Estado	CUMPLIDO
Check validado	Resultado
API responde 200	EXITOSO
respuesta incluye success=true	EXITOSO
catálogo responde 200	EXITOSO
categorías responde 200	EXITOSO

5.2 Prueba Load

La prueba load permitió validar el comportamiento del backend ante un incremento gradual de usuarios virtuales hasta un máximo de 10 VUs. Este escenario simula un uso concurrente moderado y permite observar si la API mantiene tiempos de respuesta aceptables.

Métrica	Resultado
Comando	npm run k6:load
Script	k6/load.js
Usuarios virtuales máximos	10 VUs
Duración del escenario	1 min 15 s
Checks totales	1827
Checks exitosos	1827 de 1827 (100.00%)
Checks fallidos	0 de 1827 (0.00%)
Peticiones HTTP	1827
http_req_failed	0.00%
Duración promedio	5.05 ms
p(95)	6.88 ms
Umbral p(95)	< 1200 ms
Iteraciones completadas	609
Estado	CUMPLIDO
Check validado	Resultado
GET /products 200	EXITOSO
GET /products/:id 200	EXITOSO
GET /categories 200	EXITOSO

5.3 Prueba Stress

La prueba stress evaluó el comportamiento de la API bajo una carga superior, llegando hasta 50 usuarios virtuales. Este tipo de prueba permite identificar si el sistema conserva estabilidad cuando la demanda incrementa de forma considerable.

Métrica	Resultado
Comando	npm run k6:stress
Script	k6/stress.js
Usuarios virtuales máximos	50 VUs

Duración del escenario	1 min 20 s
Checks totales	7221
Checks exitosos	7221 de 7221 (100.00%)
Checks fallidos	0 de 7221 (0.00%)
Peticiones HTTP	7221
http_req_failed	0.00%
Duración promedio	2.2 ms
p(95)	3.86 ms
Umbral p(95)	< 2000 ms
Iteraciones completadas	2407
Estado	CUMPLIDO
Check validado	Resultado
catálogo responde 200 bajo estrés	EXITOSO
detalle responde 200 bajo estrés	EXITOSO
categorías responden 200 bajo estrés	EXITOSO

6. Análisis técnico de resultados

Los resultados obtenidos evidencian que el backend mantiene un comportamiento estable y eficiente. En las pruebas Jest, todos los casos de prueba fueron exitosos y la cobertura superó el objetivo mínimo establecido. En las pruebas k6, los tres escenarios presentaron 0.00% de fallos HTTP, lo que indica que las peticiones evaluadas fueron atendidas correctamente por la API.

7. Evidencia de comandos ejecutados

7.1 Comandos Jest

- *npm run test*
- *npm run test:coverage*

Resultado registrado:

Test Suites: 6 passed, 6 total

Tests: 47 passed, 47 total

Snapshots: 0 total

Time: 3.518 s

7.2 Comandos k6

- *npm run k6:smoke*
- *npm run k6:load*
- *npm run k6:stress*

Resumen de resultados registrados:

Smoke: checks_succeeded=100.00%, http_req_failed=0.00%, p(95)=5.4ms

Load: checks_succeeded=100.00%, http_req_failed=0.00%, p(95)=6.88ms

Stress: checks_succeeded=100.00%, http_req_failed=0.00%, p(95)=3.86ms

8. Conclusión y dictamen de calidad

Con base en los resultados obtenidos, se concluye que el backend del sistema de Joyería Maison C.O. Élégance cumple satisfactoriamente con los criterios de calidad definidos para esta fase del proyecto. Las pruebas unitarias ejecutadas con Jest validaron correctamente la lógica principal del sistema, alcanzando una cobertura global superior al 90% en todas las métricas evaluadas.

Asimismo, las pruebas de rendimiento ejecutadas con k6 demostraron que la API mantiene estabilidad bajo escenarios de carga mínima, carga moderada y estrés. En los tres escenarios se obtuvo 100.00% de checks exitosos y 0.00% de fallos HTTP, cumpliendo los umbrales establecidos para tiempo de respuesta y confiabilidad.